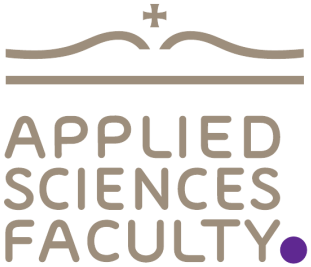
C++ — додаток. Qt. Lecture 3 — допоміжні засоби: QString, QLable, журналювання, підтримка перекладу





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Допоміжні засоби

Перш ніж рухатися далі, розглянемо ряд службових засобів Qt.

QString — 1

По перше вартує згадати про власний клас стрічок Qt[1], QString:

- Це клас, призначений зберігати Unicode-стрічки[2].
- Внутрішнє кодування QChar – UTF-16, для символів за межами base plane використовуються сурогатні пари, два послідовних QChar.
- Використовує copy-on-write.
- Сам Qt користується практично виключно QString для представлення стрічок.
- На загал, він схожий на std::string.
- Приклад конструювання:

```
QString str1a = "Hello";
QString str1b("Hello");
std::string cppstr="Hello C++";
QString str1c = QString::fromStdString(cppstr);
```

Подробиці перетворення із звичних, 8-бітних, C-стрічок розглянемо пізніше.

[1] Кожен поважний framework намагається завести собі клас стрічок.

[2] Всередині зберігає елементи типу QChars Зокрема, може містити нульові символи, QChar::Null, вони НЕ служать ознакою кінця стрічки.

QString — 2

- Метод `int size()` повертає розмір стрічки,
- Метод `void resize(int size)` змінює його. Якщо новий розмір більший за вихідний, нові літери неініціалізовані.
- Звертатися до окремих літер можна оператором operator[] та константним методом at()[3].

```
QChar qc1;
qc1 = str1a[1];
str1b[str1b.size()-1] = qc1;
qc1 = str1a.at(0);
```

Увага! Значення at() інше, ніж в STL – просто повертає копію літери, operator[] повертає QCharRef.

- Перевантажено й багато інших операторів, таких як operator==, operator!=, operator<, operator>, operator<=, operator>=, operator+=, operator+. Це – не рахуючи оператори присвоєння та вводу виводу.

[3] Так як QString користується оптимізацією Copy-on-Write, at може бути швидшим за operator[], бо ніколи не призводить до створення копії.

QString — 3

- Методи `QString QString::left(int n) const`, `QString QString::mid(int position, int n = -1) const`, `QString QString::right(int n) const`, повертають копію підстрічки з початку, середини чи кінця стрічки.
- Їхні "кузени", leftJustified, rightJustified повертають вирівняну до заданої ширини частину стрічки, а leftRef, rightRef, midRef — повертають, замість копії, посилання на підстрічку.
- Макрос QStringLiteral(str) дозволяє створювати ефективні літерали типу QString:

```
QString str = QStringLiteral("ABC");
```

- Є багато функцій-методів для маніпуляції стрічками: append(), prepend(), insert(), replace(), and remove().

```
QString str = "have";
str.prepend("I ");              // str == "I have"
str.append(" an apple.");       // str == "I have an apple."
str.replace(0, 6, "He has");    // str == "He have an apple."
```

QString — 4

- Метод trimmed() викидає whitespace-и, пробіли, табуляції, тощо, з початку і кінця стрічки.
- simplified() крім видалення всіх пробілів на початку та в кінці, також викидає і повторні пробіли всередині стрічки.
- Методи indexOf() та lastIndexOf() здійснюють пошук під-стічки з початку чи з кінця, відповідно. Повертають індекс початку підстрічки, якщо знайдено, -1 в іншому випадку.
- Методи toUpper() та toLower() перетворюють всі літери стрічки на великі чи малі, відповідно.

QString — 5

- Методи сімейства arg() дозволяють підставляти параметри на місці відповідних маркерів, виду %1, %2, і т.д. Один виклик arg підставляє свій аргумент на місце маркера з найменшим номером:

```
QString errMsg =
    QString("Error  %1 in module %2 for argument %3")
            .arg("Wrong data").arg("Main").arg("unknown");
```

- Стрічки:

```
QString arg(char a, int fieldWidth = 0, QChar fillChar = QLatin1Char(' ')) const
QString arg(QChar a, int fieldWidth = 0, QChar fillChar = QLatin1Char(' ')) const
QString arg(const QString &a, int fieldWidth = 0, QChar fillChar = QLatin1Char(' ')) cons
QString arg(QLatin1String a, int fieldWidth = 0, QChar fillChar = QLatin1Char(' ')) const
QString arg(const QString &a1, const QString &a2) const
    ...............
QString arg(const QString &a1,    ...........,    const QString &a9) const
QString arg(QStringView a, int fieldWidth = 0, QChar fillChar = QLatin1Char(' ')) const
```

- Цілі числа, де inttype – один із qulonglong, qlonglong, long, ulong, int, uint, short, ushort:

```
QString arg( inttype a, int fieldWidth = 0, int base = 10, QChar fillChar = QLatin1Char('
```

- double:

```
QString arg(double a, int fieldWidth = 0, char format = 'g', int precision = -1, QChar f
```

QString — 6

- Методи startsWith() і endsWith() дозволяють перевірити, чи стрічка починається/закінчується із вказаної під-стрічки.
- Метод contains() повертає true, якщо підстірчка є в стрічці,
- count() повертає кількість входжень.
- Статична функція `QString QString::number(long n, int base = 10)` повертає число, що відповідає стрічці, користуючись під час перетворення вказаною основою.
- Статична фукнкція `QString & QString::setNum(int n, int base = 10)` навпаки, перетворює число у відповідну стрічку.
- Колекція методів toInt(), toLong(), toDouble(), тощо, роблять саме те, що обіцяє назва (деталі див. документацію).
- Сімейство методів split та join дозволяють розбивати стрічки на список стрічок (QStringList) чи об'єднувати такий список у стрічку, в тому числі і з використанням регулярних виразів (QRegExp).

QString, взаємодія із 8-бітними стрічками — 1

- В конструкціях типу

  QString str = "have";

  повинне відбуватися перетворення із 8-бітних стрічок у Unicode, яке використовується QString.
- Конструктор, якщо йому не вказати іншого, здійснює перетворення за допомогою fromUtf8() — вихідна C-стрічка текст вважатиметься закодованою в UTF-8.
- Існують також fromLatin1() і fromLocal8Bit(), які вважають стрічку закодованою Latin-1 (ISO 8859-1) або локальним системним кодуванням, відповідно. Якщо потрібно мати справу із ще якимись кодуваннями, слід скористатися класом QTextCodec.
- Заборонити автоматичні перетворення можна, визначивши макроси QT_NO_CAST_FROM_ASCII та QT_NO_CAST_TO_ASCII. Тоді функції сімейства fromXXX доведеться викликати явно.

QString, взаємодія із 8-бітними стрічками — 2

- Аналогічно, щоб перетворити QString в масив char, можна скористатися методами toLatin1(), toUtf8(), toLocal8Bit() або все тим же QTextCodec, який ми тут не розглядатимемо.
- Для взаємодії із std::string є методи fromStdString() та toStdString(). Можуть бути відсутніми, якщо Qt скомпільовано без підтримки STL.

Детальніше див. документацію:
`https://doc.qt.io/qt-5/qstring.html`,
`https://doc.qt.io/qt-5/qtextcodec.html` тощо.

QStringRef

- Посилання на частину стрічки. **Лише для читання.**
- Метод string() повертає вказівник на оригінальну стрічку.
- size() – розмір частини, на яку посилається, а position() – її позицію у вихідній стрічці.
- toString() створює відповідну копію.
- Конструктори:

```
QStringRef(const QString *string, int position, int length)
QStringRef(const QString *string)
QStringRef(const QStringRef &other)
QStringRef(QStringRef &&other)
```

- В решті – інтерфейс схожий на QString.
- QString має такі корисні методи:

`QVector<QStringRef> QString::splitRef(...).`

Детальніше див. документацію:

`https://doc.qt.io/qt-5/qstringref.html.`

QStringView

- QStringView – "погляд" на UTF-16 стрічку, із інтерфейсом лише для читання, типу QString.
- Рекомендується передавати по значенню, а не за константним посиланням.
- QStringRef може посилатися лише на константний QString, а QStringView – на будь-який масив UTF-16 символів. З іншого боку, QStringRef "переживає" реалокацію в "його" QString.

Детальніше див. документацію:
`https://doc.qt.io/qt-5/qstringview.html#details`.

Журналювання — logging

- Файл <QtGlobal> містить ряд функцій для виводу різноманітних службових повідомлень: qCritical(), qDebug(), qFatal(), qWarning().
- Виводом займається певна функція, message handler.
- Поведінка цієї функції по замовчуванню залежить від системи. По замовчуванню пишуть в stderr, під Windows — передають текст відладчику.
- Виклик цих функцій схожий на виклик функцій сімейства printf.
- Приклад:

```
if (ScreenSize < 200)
    qWarning("Too small screen size (%d).", ScreenSize);
```

- Якщо включити також <QtDebug>, можна буде писати так[4]:

```
if (ScreenSize < 200)
    qWarning() << "Too small screen size (" << ScreenSize << ")";
```

[4] Крім qFatal() — для нього цей синтаксис не підтримується

Керування журналюванням

- Макроси QT_NO_WARNING_OUTPUT та QT_NO_DEBUG_OUTPUT можуть заборонити вивід відповідних повідомлень.
- Змінна середовища QT_MESSAGE_PATTERN дозволяє керувати форматом повідомлення:

  ```
  QT_MESSAGE_PATTERN="[%{type}] %{appname} (%{file}:%{line}) - %{mes
  ```

- Свою реакцію на відповідні повідомлення можна встановити за допомогою функції qInstallMessageHandler:

  QtMessageHandler qInstallMessageHandler(QtMessageHandler handler);

  де переданий аргумент — вказівник на функцію із такою сигнатурою:

  void myMessageHandler(QtMsgType, **const** QMessageLogContext &, **const** QString &);

- Якщо передано нульовий вказівник, встановлює
- Детальніше про значення аргументів див. `http://qt-project.org/doc/qt-5/debug.html`, а ми подивимося на приклад:

Приклад — встановлюємо:

```
if (our_handler)
{
      qInstallMessageHandler (0);
      our_handler = false;
      setRestoreHandlerButton->setText(setHandlerText);
}else
{
     qInstallMessageHandler(ourMessageHandler);
     our_handler = true;
     setRestoreHandlerButton->setText(resetHandlerText);
}
```

Приклад — реалізовуємо:

```
void ourMessageHandler(QtMsgType type,
        const QMessageLogContext &context,
        const QString &msg)
{
    switch (type) {
    case QtDebugMsg:
    case QtWarningMsg:
    case QtCriticalMsg:
      logger->appendOutput( msg + QString(" At file %2:%3, %4)\n").
            arg(context.file ).arg(context.line ).arg(context.function )  );
        break;
    case QtFatalMsg:
      logger->appendOutput( msg + QString(" Fatal at file %2:%3, %4)\n")
            arg(context.file ).arg(context.line ).arg(context.function )  );
      qApp->exit(1);
    }
}
```

Детальніше див. проект `qtl03_MsgHandler`.

Сприяння перекладу

- Qt надає розвинуті засоби для локалізації — перекладу програм на різноманітні мови.
- Першим етапом підготовки програми до локалізації є включення всіх стрічок, які може виникнути потреба перекласти, за допомогою функції tr():

```
quitMsg.setWindowTitle( tr("Hello, world!") );
quitMsg.setText( tr("Do you really want to quit?") );
```

- Надалі ми всі стрічки задаватимемо саме так.
- Так як переклад конкретного слова чи речення часто сильно залежить від контексту, можна створювати коментарі для перекладачів, використовуючи `//: ...` або `/*: ... */` безпосередньо перед стрічкою:

```
//: Quit confirmation dialog title.
quitMsg.setWindowTitle( tr("Hello, world!") );
```

Документація, що стосується перекладу

Детальніше переклад ми тут не розглядатимемо, за додатковою інформацією див. наступні джерела:

- `https://doc.qt.io/qt-5/qtlinguist-hellotr-example.html` — "Hello tr() Example"
- `https://doc.qt.io/qt-5/internationalization.html` — "Internationalization with Qt"
- `https://doc.qt.io/qt-5/qtlinguist-index.html` — "Qt Linguist Manual"
- `https://doc.qt.io/qt-5/i18n-source-translation.html` — "Writing Source Code for Translation"

Додатково про QLabel

Насправді, клас QLabel дуже потужний. Окрім звичайного тексту, він вміє виводити:

- Форматований текст (RichText).
- Зображення.
- Анімацію.
- Числа.

Коли встановлюється той чи інший вміст QLabel, попередній вміст очищається.

Текстові мітки

- Форматування тексту, згаданого вище, здійснюється підмножиною тегів HTML4. По замовчуванню, якщо теги є — текст виводиться як форматований, інакше — як звичайний.
- Встановити текст можна як під час створення мітки, так і за допомогою методу setText():

```
plain_label->setText("Plain text\nThough, multi-line.");
```

- Метод setTextFormat() дає можливість примусово виводити текст як форматований, чи навпаки.

```
plain_label_with_HTML->setTextFormat(Qt::PlainText);
```

можливі значення: Qt::RichText, Qt::PlainText, Qt::AutoText — автоматичний вибір.

- Метод setWordWrap() дозволяє включити перенесення по словах у мітці:

```
rich_label->setWordWrap(true);
```

Частина HTML, що підтримується, описана тут:
http://qt-project.org/doc/qt-5/richtext-html-subset.html

Вирівнювання

- Вирівнюванням вмісту мітки можна керувати за допомогою методу setAlignment(), який приймає значення з перерахування Qt::Alignment:

```
label->setAlignment(Qt::AlignBottom | Qt::AlignRight);
```

- Окремо можна керувати вирівнюванням по горизонталі та по вертикалі. Побітовим | можна комбінувати одне із можливих горизонтальних та одне із можливих вертикальних видів вирівнювання.
- Вирівнювання по горизонталі: Qt::AlignLeft, Qt::AlignRight, Qt::AlignHCenter, Qt::AlignJustify.
- Вирівнювання по вертикалі: Qt::AlignTop, Qt::AlignBottom, Qt::AlignVCenter, Qt::AlignBaseline.
- Існує одне "двомірне" вимірювання: Qt::AlignCenter, яке є еквівалентним `AlignVCenter | AlignHCenter`.

"Рамки"-фрейми

Для наочності, хотілося б бачити границі міток. Мітка є нащадком QFrame, та успадкувала від нього метод: setFrameStyle(), за допомогою якого можна задати стиль будь-якого фрейма:

```
plain_label->setFrameStyle(QFrame::Panel | QFrame::Sunken);
```

Стилі складаються із двох типів:

- Стилі тіней: QFrame::Plain, QFrame::Raised, QFrame::Sunken;
- та стилі форми: QFrame::NoFrame, QFrame::Box, QFrame::Panel, QFrame::StyledPane, QFrame::HLine, QFrame::VLine, QFrame::WinPanel.
- Комбінувати їх можна побітовим оператором OR — |.

Детальніше див.
`http://qt-project.org/doc/qt-5/qframe.html`

Розміри — 1

- Від QWidget мітка успадковує цілу колекцію методів керування розміром:

```
void    setMinimumHeight(int minh);
void    setMinimumWidth(int minw);
void    setMinimumSize(const QSize &);
void    setMinimumSize(int minw, int minh);

void    setMaximumHeight(int maxh);
void    setMaximumWidth(int maxw);
void    setMaximumSize(const QSize &);
void    setMaximumSize(int maxw, int maxh);

void     resize (int  w,  int  h );
void     resize (const QSize &);
```

+ setFixedXXX, які встановлюють максимальні і мінімальні розміри одночасно. QSize — простий клас для збереження розміру (ширини та висоти) тієї чи іншої видимої сутності, детальніше див. далі.

Розміри — 2

- Перевірити можна відповідними "getter" методами:

```
int     minimumHeight() const;
int     minimumWidth() const;
QSize   minimumSize() const

int     maximumHeight() const;
int     maximumWidth() const;
QSize   maximumSize() const;

QSize   size () const;
int     height () const;
int     width() const;
```

Вони повертають розмір без врахування фрейма-рамки, якщо така існує.

Розміри — 3

- Крім описаних вище прямолінійних методів керування розміром, є ще декілька хитріших. Поміж них, наприклад:
- **void** adjustSize ();

  налаштовує розміри згідно певних правил. Детальніше див. документацію на QWidget, а якщо коротко, то він намагається орієнтуватися на sizeHint(), якщо там нулі, то змінює розмір так, щоб вмістити всіх нащадків.
- sizeHint(), у свою чергу, є представником цілого сімейства hint-методів, які надають *підказки* щодо оптимального, мінімального та максимального розмірів об'єктів. Детальніше поки їх не розглядатимемо.

УВАГА! Коли віджетом керує layout, він може ігнорувати встановлений resize розмір! Інша справа, що Maximal і Minimal розміри намагається поважати. Детальніше див. наступні лекції або документацію на QLayout і його нащадків.

Як би там не було, маніпуляція розмірами вимагає великої акуратності.

QSize

- Клас QSize зовсім простий, щоб його детально описувати, тому лише наведу прототипи його методів:

```
QSize();
QSize(int width, int height);
QSize    boundedTo(const QSize & otherSize) const;
QSize    expandedTo(const QSize & otherSize) const;
int      height() const;          int &      rheight();
int      width() const;           int &      rwidth();
void     setHeight(int height);   void       setWidth(int width)
bool     isEmpty() const;
bool     isNull() const;
bool     isValid() const;
void     scale(int width, int height, Qt::AspectRatioMode mode);
void  scale(const QSize & size, Qt::AspectRatioMode mode);
QSize scaled(int width, int height, Qt::AspectRatioMode mode) const;
QSize    scaled(const QSize & s, Qt::AspectRatioMode mode) const;
void     transpose();             QSize      transposed() const;
```

- Крім того, перевантажено арифметичні оператори, +=, -=, *=, /=, +, -, *, /, оператори ==, != та << і >>.

Детальніше див. `http://qt-project.org/doc/qt-5/qsize.html`

Розміри — QWidget і нащадки!

УВАГА! Хоча вище керування розмірами ми розглядали на прикладі *QLabel*, пам'ятайте, що інтерфейс цей походить з **QWidget** і є у всіх його нащадків!

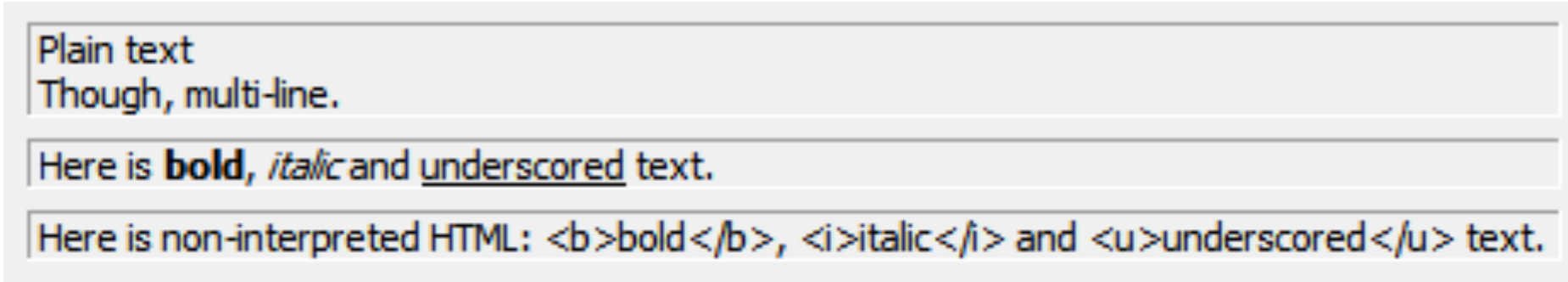
```
QLabel *plain_label = new QLabel;
plain_label->setText("Plain text\nThough, multi-line.");
plain_label->setFrameStyle(QFrame::Panel | QFrame::Sunken);

QLabel *rich_label = new QLabel;
rich_label->setText("Here is <b>bold</b>, <i>italic</i> \
                     and <u>underscored</u> text.");
rich_label->setFrameStyle(QFrame::Panel | QFrame::Sunken);

QLabel *plain_label_with_HTML = new QLabel;
plain_label_with_HTML->setText("Here is non-interpreted \
                     HTML: <b>bold</b>, <i>italic</i> \
                     and <u>underscored</u> text.");
plain_label_with_HTML->setFrameStyle(QFrame::Panel | QFrame::Sunken);
plain_label_with_HTML->setTextFormat(Qt::PlainText);

QVBoxLayout *layout = new QVBoxLayout;

layout->addWidget(plain_label);
layout->addWidget(rich_label);
layout->addWidget(plain_label_with_HTML);
```


Plain text
Though, multi-line.
Here is bold, italic and underscored text.
Here is non-interpreted HTML: bold, italic and <u>underscored</u> text.

Виведення зображень

- Сам по собі текст не завжди цікавий. Картинка може бути веселішою.
- QLable має метод setPixmap(), який встановлює зображення для виводу.
- setPixmap() приймає об'єкт класу QImage.

Класи для роботи із зображеннями:

- QImage — призначений для вводу-виводу зображень, надає доступ до окремих пікселів та дозволяє маніпуляції над зображенням. Може служити полотном для малювання (canvas-ом), так як є нащадком QPaintDevice. QPaintDevice це такий клас, на якому може малювати клас QPainter. Їх ми розглянемо трішки пізніше.
- QPixmap — оптимізований для виведення зображення на екран. Доступ до окремих пікселів утруднено — хіба за допомогою QPainter, (QPixmap теж є нащадком QPaintDevice), або перетворивши спочатку в QImage. Однак можливість завантажити зображення із файлу присутня.
- QBitmap — є монохромним (1-бітним) варіантом QPixmap.
- QPicture — призначений зберігати та відтворювати команди малювання QPainter.
- По замовчуванню підтримуються наступні формати зображень: BMP, GIF, JPG/JPEG, PNG, PBM, PGM, PPM, XBM, XPM.
- За допомогою плагінів список можна розширювати. Список підтримуваних форматів можна отримати динамічно, функціями QImageReader::supportedImageFormats() і QImageWriter::supportedImageFormats().

Поки для наших потреб цілком достатньо QPixmap:

```
QLabel *image_label = new QLabel;
QPixmap pixmap1("../Project03/daniela_uhlig_seemonster.jpg");

image_label->setAlignment(Qt::AlignCenter);
image_label->setFrameStyle(QFrame::Panel | QFrame::Sunken);
image_label->setPixmap(pixmap1);
```

Plain text
Though, multi-line.

Here is **bold**, *italic* and underscored text.

Here is non-interpreted HTML: bold, italic and <u>underscored</u> text.



Виведення відео та анімації

- QLabel вміє виводити також просту анімацію (типу GIF чи MPEG).
- Для виведення анімації слід скористатися методом setMovie(), якому передається вказівник на об'єкт QMovie. **Увага!**, що нетипово, володіння об'єктом QMovie **не передається** мітці.
- Підтримувані формати можна отримати за допомогою функції QMovie::supportedFormats().
- Вивід звуку не підтримується — для цього є більш потужні класи, з Qt Multimedia framework.

Детальніше див. документацію,
`http://qt-project.org/doc/qt-5/qmovie.html` і
`http://qt-project.org/doc/qt-5/qtmultimedia-index.html`.

Виведення чисел

Якщо викликати метод QLabel::setNum(...), передавши йому int або double, він виведе відповідне число.

```
QLabel *int_label = new QLabel;
int_label->setNum(377);
int_label->setFrameStyle(QFrame::Panel | QFrame::Sunken);

QLabel *double_label = new QLabel;
double_label->setNum(3.1415);
double_label->setFrameStyle(QFrame::Panel | QFrame::Sunken);
```

Але цей спосіб, через неможливість контролювати форматування — "для лінивих", застосовний лише в найпростіших випадках.

buddy

- QLabel вміє ще одну корисну річ — він здатен переключати фокус на вказаний віджет.
- Скажімо, якщо на формі є текстове поле, біля якого мітка-підпис, можна зробити гарячу клавішу для мітки та вказати, який віджет має отримати фокус за виклику цієї мітки:

```
QLineEdit *someEdit = new QLineEdit(this);
QLabel *someEditLabel = new QLabel("E&nter here:", this);
someEditLabel->setBuddy(someEdit);
```